

Оптимизация работы приложения

В прошлом уроке мы научились управлять двигателями машинки: написали драйвер управления и настроили общение сервера и ESP32, чтобы подавать управляющие воздействия через python-скрипт. Осталась одна проблема: одновременная работа камеры и отправка кадров на сервер приводила к большим задержкам при управлении движением машинки. Сегодня мы возьмемся исправить данную проблему - оптимизируем логику работы программы.

1. Объяснение проблемы

В первоначальной версии программы управление моторами и отправка видеок кадров выполнялись последовательно в одном цикле `loop()`. Это приводило к тому, что во время передачи JPEG-изображения на сервер (что могло занимать от 100 мс до 5 секунд) робот переставал реагировать на команды с сервера. Требовалось обеспечить параллельную работу: отправка видео не должна блокировать управление двигателями. Данную проблему мы можем решить, используя FreeRTOS.

2. Многозадачность во FreeRTOS

RTOS (Real-Time Operating System) — это операционная система реального времени. Её главная особенность в том, что она гарантирует выполнение задач в заданные временные рамки. Микроконтроллер ESP32 имеет два ядра. С помощью операционной системы реального времени FreeRTOS можно закрепить разные задачи за разными ядрами, добившись истинной параллельности.

В простых скетчах Arduino весь код выполняется последовательно в цикле `loop()`, что у нас происходило ранее в рамках курса. Приложение выполняло одну конкретную задачу, что не приводило к проблемам с работоспособностью программы. В данный момент мы столкнулись с проблемой что если внутри цикла происходит долгая операция (например, отправка JPEG-файла весом 10 КБ по Wi-Fi), микроконтроллер не может выполнять другие действия — управление моторами «замирает» на несколько секунд.

Для начала опишем основные понятия RTOS:

1. **Задача (task)** — это бесконечный цикл, который выполняет определенную работу (например, опрос датчиков, отставку данных,

управление двигателями). Каждая задача имеет свой стек и приоритет.

2. **Планировщик** (scheduler) — ядро RTOS, которое решает, какая задача будет выполняться в данный момент. Планировщик может вытеснять (прерывать) низкоприоритетную задачу, если готова высокоприоритетная.
3. **Приоритет** — целое число, указывающее важность задачи. Чем выше приоритет, тем чаще задача получает процессорное время. Задача с приоритетом 3 всегда будет прерывать задачу с приоритетом 1.
4. **Мьютекс** (mutex) — механизм синхронизации, который защищает общий ресурс (например, Wi-Fi клиент) от одновременного доступа из разных задач.
5. **Очереди** (queue) — способ обмена данными между задачами.
6. **Задержка** (delay) в RTOS реализуется через `vTaskDelay()`, которая не блокирует процессор, а переключает планировщик на другие задачи.

3. Модификация приложения

В исходной программе все функции (**`cameraHandler()`** и **`driverHandler()`**) вызывались по очереди внутри **`loop()`**. Отправка кадра занимала до 5 секунд, и всё это время робот не реагировал на команды. Чтобы решить эту проблему, мы переместим код из двух обработчиков в задачи. Для начала нужно подключить к проекту (основной файл) файл, который нам позволит создавать задачи и настраивать их:

```
#include <esp_task_wdt.h> // Для работы с потоками
```

Теперь создадим две функции-задачи:

```
void taskDriver(void *pvParameters)
{

}

void taskCamera(void *pvParameters)
{

}
```

Данные функции будут вызываться **планировщиком** на разных ядрах микроконтроллера, осуществляя параллельную работу. Настроим их вызов с помощью функции `xTaskCreatePinnedToCore` в функции `setup()`:

// Задача управления моторами (ядро 0, приоритет 2 – чуть выше камеры)

```
xTaskCreatePinnedToCore(  
    taskDriver, // функция задачи  
    "MotorDriver",  
    4096,       // стек (байт)  
    NULL,  
    2,         // приоритет  
    NULL,  
    0         // ядро 0  
);
```

// Задача отправки кадров (ядро 1, приоритет 1)

```
xTaskCreatePinnedToCore(  
    taskCamera, // функция задачи  
    "CameraSend",  
    8192,       // увеличенный стек для работы с буфером кадра  
    NULL,  
    1,         // приоритет  
    NULL,  
    1         // ядро 1  
);
```

Как можно увидеть, мы указываем **не только ядро**, которое будет использовать задача, но и **задаем приоритет** вызова функций. В данном случае приоритет, указанный при создании задачи, не влияет на работу функций, т.к. они работают на разных ядрах, но, если вы будете в будущем на одном ядре вы выполнять **несколько задач**, то **чем выше число (от 1 до 24)**, тем задача будет **приоритетнее**, следовательно, при одновременном вызове двух задач будет исполняться более приоритетная задача. Так же в качестве параметров мы передаем **функции**, которые будут вызываться планировщиком, задаем **имя задачи** и **выделяем стек** для задачи. **Стек** нужен для того, чтобы хранить контекст (состояние) задачи, т.к. планировщик может вытеснить задачу в любой момент. Чтобы позже восстановить её выполнение, необходимо сохранить все её локальные переменные и адрес следующей инструкции. Это и есть контекст задачи,

который хранится в её стеке. С настройкой задач закончили, пойдем дальше.

Следующее, что нам необходимо добавить - это **Watchdog Timer**. Это специальный таймер, который следит за тем, чтобы наша программа не “зависала”. Каждую итерацию цикла мы сбрасываем этот таймер, чтобы он не переполнился. Если программа зависнет, то мы не сможем сбросить таймер, что приведет к перезагрузке платы, и программа начнет свою работу заново. Приступим. После настройки задач добавьте следующий код:

```
esp_task_wdt_config_t wdt_config = {  
    .timeout_ms = 10000,  
    .trigger_panic = false, // просто перезагрузить  
};  
esp_task_wdt_init(&wdt_config);  
esp_task_wdt_add(NULL);
```

В данном отрывке мы задаем, что таймер будет перезагружать плату после 10 секунд простоя. В функции **loop()** поставьте сброс таймера, чтобы он не перезагружал плату каждые 10 секунд:

```
esp_task_wdt_reset(); // сообщаем, что всё работает
```

```
// Пусто – всё обрабатывается в задачах  
vTaskDelay(pdMS_TO_TICKS(1000));
```

Так же поставьте задержку для основного цикла, чтобы он не нагружал процессор - это стандартная практика, используемая при написании программ для микроконтроллеров. Пустой цикл может приводить к некорректной работе приложения.

Последнее, что нам осталось добавить - это **мьютекс**. **Мьютекс** — это специальный объект синхронизации, который позволяет организовать доступ к разделяемому ресурсу так, чтобы в каждый момент времени только одна задача могла его использовать. По сути, это «замок» или «ключ»: задача, которая хочет работать с общим ресурсом, должна сначала «захватить» мьютекс, а после окончания работы — «освободить» его. Он нам нужен, т.к. **если две задачи одновременно будут вызывать**

wifi.send() и **wifi.request()**, то пакеты HTTP перепутаются, соединение разрушится, а ESP32 может зависнуть или перезагрузиться. Создайте глобальную переменную:

```
// Мьютекс для доступа к WiFi (разделяемый ресурс)
SemaphoreHandle_t wifiMutex = NULL;
```

Теперь в функции **setup()** создадим мьютекс перед кодом, создающим задачи:

```
// Создаём мьютекс для защиты WiFi
wifiMutex = xSemaphoreCreateMutex();
if (wifiMutex == NULL) {
    Serial.println("Не удалось создать мьютекс");
    return;
}
```

Создайте еще одну глобальную переменную, которая будет регулировать частоту кадров:

```
TickType_t frameInterval = pdMS_TO_TICKS(Camera::captureInterval);
```

Теперь осталось заполнить функции задач кодом и все готово. Начнем с **taskDriver()**:

```
// Задача управления моторами (ядро 0, приоритет 3)
void taskDriver(void *pvParameters) {
    while (true) {
        // Если Wi-Fi подключен
        if (wifi.wifiCheck()) {
            // Захватываем мьютекс с таймаутом 1 секунда
            if (xSemaphoreTake(wifiMutex, pdMS_TO_TICKS(1000)) == pdTRUE) {
                // Считываем запрос сервера
                wifi.request();
                // Проверяем, пришла ли новая команда с сервера на управление двигателем
                if (wifi.isNeedUpdate()) {
                    // Получаем команду
                    WifiManager::RobotCommands commands = wifi.getCommand();
                    // Управляем моторами
                    driver.move(commands.leftMotor, commands.rightMotor);
                }
            }
        }
    }
}
```

```

        xSemaphoreGive(wifiMutex);
    }
}
vTaskDelay(pdMS_TO_TICKS(200));    // опрос каждые 200 мс
}
}

```

Задача **taskDriver** отвечает за связь с сервером и управление двигателями робота. Она работает в бесконечном цикле, периодически опрашивая сервер на наличие новых команд и мгновенно обрабатывая их.

```
if (wifi.wifiCheck())
```

Перед тем как отправлять запрос к серверу, задача убеждается, что Wi-Fi соединение установлено. Если соединение потеряно, блок кода пропускается.

```
if (xSemaphoreTake(wifiMutex, pdMS_TO_TICKS(1000)) == pdTRUE)
```

Мьютекс (wifiMutex) защищает доступ к Wi-Fi, который одновременно используется и задачей камеры (для отправки кадров). Функция **xSemaphoreTake** пытается захватить мьютекс. Параметр **pdMS_TO_TICKS(1000)** означает, что задача будет ждать освобождения мьютекса не более 1 секунды. Если мьютекс захвачен успешно (pdTRUE), задача может безопасно использовать Wi-Fi. Если нет – она пропускает текущий цикл и переходит к задержке. Это предотвращает бесконечное ожидание и сохраняет отзывчивость системы.

```
wifi.request();
```

Через объект wifi (класс WifiManager) отправляется GET-запрос на сервер по пути /getdata. Сервер возвращает строку с текущими командами, например LEFT:1,RIGHT:-1.

```
if (wifi.isNeedUpdate())
```

Метод **isNeedUpdate()** возвращает true, если с предыдущего вызова **wifi.request()** пришли новые данные. Это позволяет избежать лишнего обновления моторов, когда команда не изменилась.

```
WifiManager::RobotCommands commands = wifi.getCommand();  
driver.move(commands.leftMotor, commands.rightMotor);
```

Структура **RobotCommands** содержит значения для левого и правого мотора в диапазоне от -1 до 1 (где -1 – полный назад, 0 – стоп, 1 – полный вперед). Метод **driver.move()** передаёт эти значения драйверу двигателей, который формирует соответствующие ШИМ-сигналы.

```
xSemaphoreGive(wifiMutex);
```

После завершения всех операций с Wi-Fi мьютекс освобождается, чтобы другие задачи (например, **taskCamera**) могли получить доступ к сети.

```
vTaskDelay(pdMS_TO_TICKS(200));
```

Функция **vTaskDelay** переводит задачу в режим ожидания на 200 миллисекунд. В это время процессор может выполнять другие задачи (например, отправку кадров). Период опроса 200 мс выбран как компромисс между быстрой реакцией на команды и снижением нагрузки на сервер и сеть.

Теперь займемся **taskCamera()**:

```
// Задача камеры и отправки кадров (ядро 1, приоритет 1)
```

```
void taskCamera(void *pvParameters) {  
    while (true) {  
        // Получение изображения с камеры  
        camera_fb_t* fb = camera.getCupture();  
        // Если изображения нет, то выходим из функции  
        if (!fb) {  
            Serial.println("Ошибка захвата кадра");  
            return;  
        }  
  
        // Увеличение счетчика кадров  
        frameCount++;  
        wifi.frameCount = frameCount;  
  
        // Отправка изображения  
        if (wifi.wifiCheck()) {
```

```

    if (wifi.isNeedUpdate()) {
        // Получаем команду
        WifiManager::RobotCommands commands = wifi.getCommand();
        frameInterval = pdMS_TO_TICKS(commands.cameraInterval);
    }
    // Захватываем мьютекс перед отправкой
    if (xSemaphoreTake(wifiMutex, pdMS_TO_TICKS(2000)) == pdTRUE) {
        wifi.send(WifiManager::SendFormat::CAMERA_DATA, fb);
        xSemaphoreGive(wifiMutex);
    }
}
// ОБЯЗАТЕЛЬНО освобождаем буфер
esp_camera_fb_return(fb);
// Задержка между кадрами (10 fps)
vTaskDelay(frameInterval);
}
}

```

Задача **taskCamera** отвечает за захват изображений с камеры, их отправку на сервер и динамическое изменение частоты кадров по команде с сервера. Работает в бесконечном цикле, выполняясь на отдельном ядре (ядро 1).

```
camera_fb_t* fb = camera.getCupture();
```

Метод **getCupture()** класса Camera возвращает указатель на структуру **camera_fb_t**, содержащую буфер с JPEG-изображением и его размер. Если кадр не получен (например, камера не инициализирована или нет памяти), возвращается **nullptr**, и задача выводит ошибку и завершается.

```

if (wifi.wifiCheck())
...
esp_camera_fb_return(fb);

```

Если соединение отсутствует, отправка пропускается, но кадр всё равно освобождается, чтобы не течь память.

```

if (wifi.isNeedUpdate()) {
    WifiManager::RobotCommands commands = wifi.getCommand();
    frameInterval = pdMS_TO_TICKS(commands.cameraInterval);
}

```


Перед отправкой кадра задача проверяет, пришла ли новая команда от сервера (через **wifi.isNeedUpdate()**). Если да, то из команды извлекается значение **cameraInterval** (в миллисекундах, задаётся пользователем на веб-странице) и преобразуется в тики **FreeRTOS** с помощью **pdMS_TO_TICKS()**. Это значение определяет задержку между кадрами. Таким образом, частота съёмки может меняться на лету без перезагрузки ESP32.

```
if (xSemaphoreTake(wifiMutex, pdMS_TO_TICKS(2000)) == pdTRUE) {  
    wifi.send(WifiManager::SendFormat::CAMERA_DATA, fb);  
    xSemaphoreGive(wifiMutex);  
}
```

Поскольку задача управления моторами тоже использует Wi-Fi (для запроса команд), необходимо синхронизировать доступ к сетевому интерфейсу. Мьютекс **wifiMutex** защищает разделяемый ресурс. Функция **xSemaphoreTake** пытается захватить мьютекс с таймаутом 2 секунды. Если мьютекс занят (например, задача **taskDriver** выполняет **wifi.request()**), задача камеры ждёт не более 2 секунд. Если захват удался, вызывается **wifi.send()** для отправки JPEG-кадра на сервер через POST-запрос. После отправки мьютекс освобождается.

Таймаут 2 секунды выбран больше, чем у задачи управления (1 секунда), так как отправка кадра может занимать больше времени, и мы не хотим, чтобы задача камеры постоянно пропускала попытки.

```
esp_camera_fb_return(fb);
```

Буфер, выделенный драйвером камеры, необходимо вернуть, иначе память быстро исчерпается, и камера перестанет захватывать новые кадры. Функция **esp_camera_fb_return** освобождает буфер.

```
vTaskDelay(frameInterval);
```

Вместо **delay()** используется **vTaskDelay()** – функция **FreeRTOS**, которая переводит задачу в состояние ожидания на указанное количество тиков. В это время процессор может выполнять другие задачи (например, **taskDriver**). Это энергоэффективно и позволяет планировщику правильно

распределять время. Значение **frameInterval** обновляется динамически при получении новой команды с сервера.

Программа закончена, проверяем ее через сервер.

4. Проверка работы

Чтобы начать работать с кодом, через Web Interface, нужно сначала войти в консоль и вбить команду **ipconfig**.

```
Настройка протокола IP для Windows

Неизвестный адаптер harp-tun:

DNS-суффикс подключения . . . . . :
Локальный IPv6-адрес канала . . . : fe80::9b1:3f83:64b1:937f%28
IPv4-адрес. . . . . : 172.18.0.1
Маска подсети . . . . . : 255.255.255.252
Основной шлюз. . . . . : 172.18.0.2

Адаптер беспроводной локальной сети Подключение по локальной сети* 9:

Состояние среды. . . . . : Среда передачи недоступна.
DNS-суффикс подключения . . . . . :

Адаптер беспроводной локальной сети Подключение по локальной сети* 10:

Состояние среды. . . . . : Среда передачи недоступна.
DNS-суффикс подключения . . . . . :

Адаптер беспроводной локальной сети Беспроводная сеть:

DNS-суффикс подключения . . . . . :
Локальный IPv6-адрес канала . . . : fe80::de33:bee0:9821:17d%10
IPv4-адрес. . . . . : 192.168.31.20
Маска подсети . . . . . : 255.255.255.0
Основной шлюз. . . . . : 192.168.31.1
```

На скриншоте выделен IP, который вы должны вставить в код в выделенную область:

```
class WifiManager {
private:
    // Настройки Wi-Fi сети
    // const char* ssid = "ИМЯ_ВАШЕЙ_СЕТИ";      // Название Wi-Fi сети
    // const char* password = "ПАРОЛЬ_СЕТИ";      // Пароль от Wi-Fi

    const char* ssid = "techDesignPr";      // Название Wi-Fi сети
    const char* password = "Vozneslab";      // Пароль от Wi-Fi

    // Адрес сервера
    const char* serverIP = "192.168.14.183"; // Адрес вашего сервера
}
```

Потом запустите через консоль python-скрипт [serverControl.py](#). У вас появится следующий текст в консоли:

```
=====
🤖 Сервер управления роботом ESP32-CAM
=====

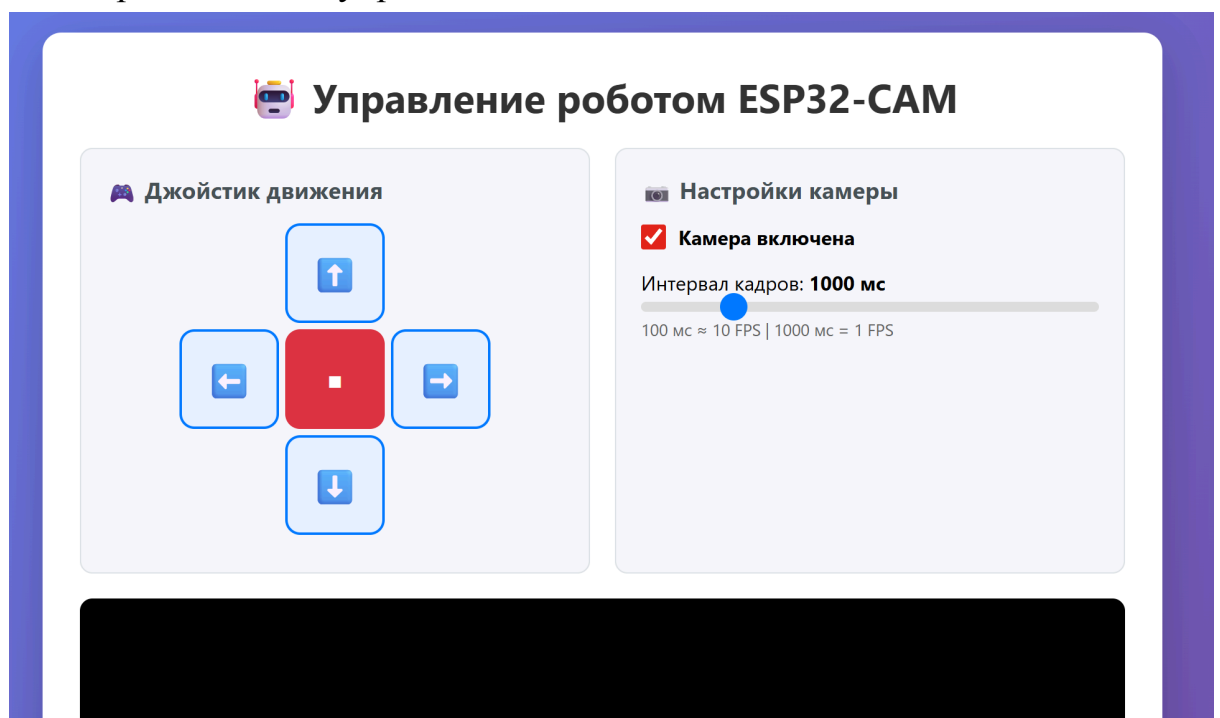
⚡ Изображения НЕ сохраняются на диск (только в RAM)

Управление:
🌐 Веб-интерфейс: http://localhost:8080/control
📺 Видеопоток: http://localhost:8080/latest
📶 Статус: http://localhost:8080/status

API команды:
GET /api/move?left=1&right=1
GET /api/camera?enabled=1&interval=500

Горячие клавиши на странице /control:
↑↓←→ - движение
Пробел - стоп
=====
|
```

Выделенный текст - это ссылка на сайт, который реализует управление машинкой через элементы управления.



Через “стрелочки” на сайте или клавиатуры можете управлять движением машинки.